

Introduction to Neural Networks and Perceptron

AI for Beginners Course

Introduction to Neural Networks

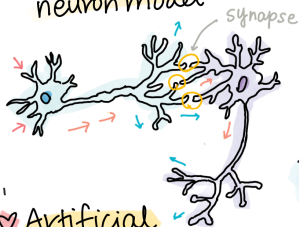
Introduction to Neural Networks

@girlie_mac
@shwars



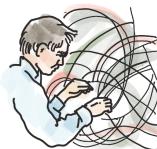
Neural Networks
↳ Deep Learning

♥ Biological
neuron model



Perception

= Single-layer
neural network
1957 Frank Rosenblatt

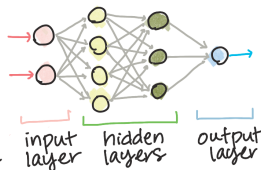


Mark-I
Perception
computer

- 400 inputs
- 1 binary output

Recognize
Primitive
Shapes

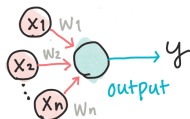
↔ Multi-layer
perception



input layer hidden layers output layer

Back propagation
method to train

♥ Artificial



$$y = f\left(\sum_{i=1}^n x_i w_i\right)$$

Frameworks

Tensor
Flow



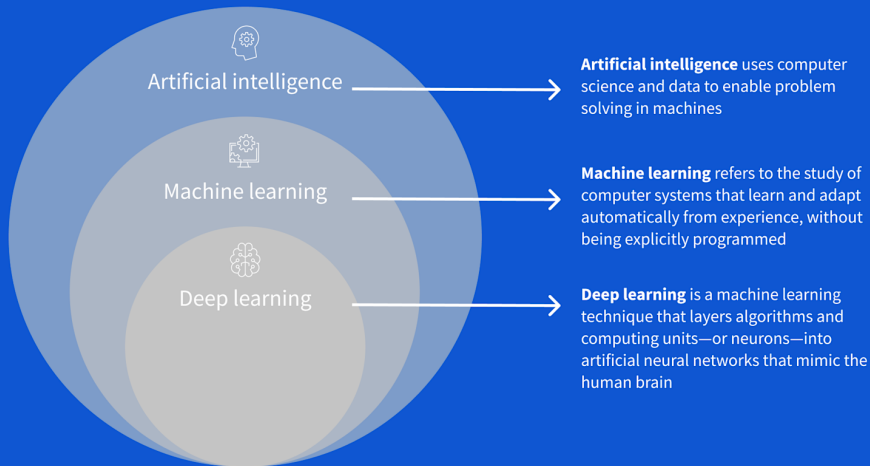
Low-level
APIs

- build
Computational
graph

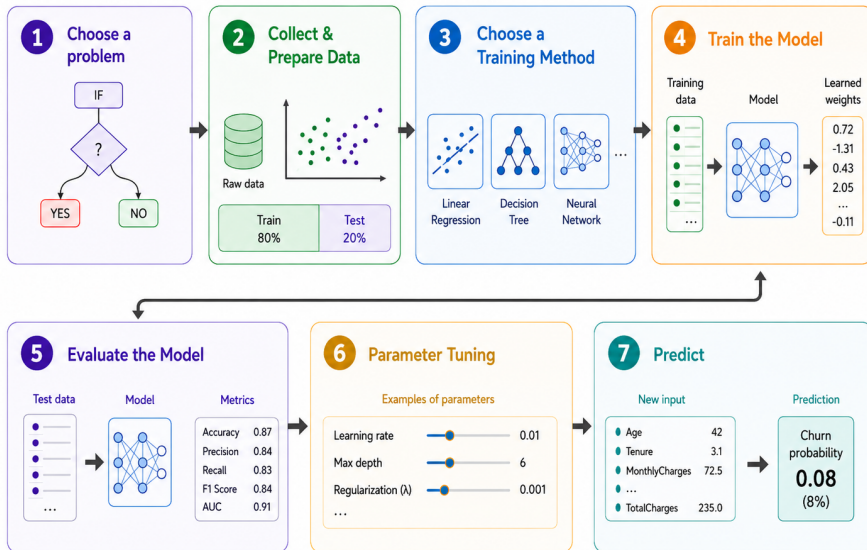


High-level APIs

AI, ML, and DL



The Craft of Creating AI/ML/DL



What are Neural Networks?

Achieving Intelligence through Training

- Train a **computer model** or **artificial brain**
- Mathematical models researched since mid-20th century
- Recently proved hugely successful
- Also called **Artificial Neural Networks** (ANNs)

Key Distinction:

- We're talking about mathematical models
- Not real biological networks of neurons

Neural Networks are part of Machine Learning

- Machine Learning: Use data to train computer models
- Large part of Artificial Intelligence
- Classical ML not covered in this curriculum

Visit **Machine Learning for Beginners** curriculum to learn more about classic Machine Learning

Basic Setup:

- Dataset of examples: $\mathbf{X}$
- Corresponding output values: $\mathbf{Y}$
- Examples: N-dimensional vectors (features)
- Outputs: Labels

Data Representation:

- Input dataset: Matrix of size M times N
 - M = number of samples
 - N = number of features
- Output labels $\mathbf{Y}$: Vector of size M

Two Main ML Problems

1. Classification

- Classify input object into 2+ classes
- Example: Is this email spam or not?
- Example: Recognize handwritten digits (0-9)

2. Regression

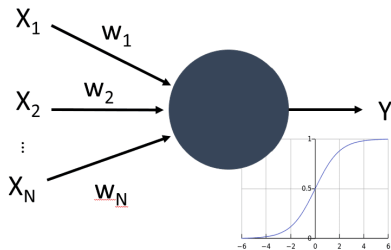
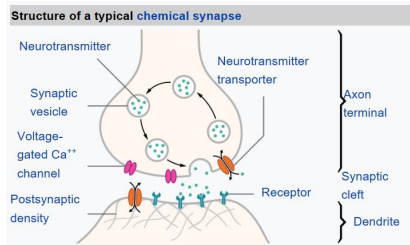
- Predict a numerical number
- Example: Predict house prices
- Example: Forecast temperature

Our Focus: Neural network models for both problems

The Human Brain:

- Consists of neural cells (neurons)
- Multiple “inputs”: **dendrites**
- Single “output”: **axon**
- Connections: **synapses**
 - Varying degrees of conductivity
 - Regulated by neurotransmitters
- Both dendrites and axons conduct electrical signals

Real vs. Artificial Neuron



Left: Real Neuron (Image from Wikipedia)

Right: Artificial Neuron (Simplified mathematical model)

Mathematical Model of a Neuron

Components:

- Multiple inputs: $X_1, X_2, \dots, X_N$
- Single output: Y
- Series of weights: $W_1, W_2, \dots, W_N$
- Activation function: f

Output Calculation:

$$Y = f \left(\sum_{i=1}^N X_i \cdot W_i \right)$$

$$Y = f(\text{sum of } X_i * W_i \text{ for all } i)$$

The Neuron Formula

Mathematical Expression:

- Weighted sum of inputs
- Applied non-linear activation function

Why Non-linear?

- Enables learning complex patterns
- Linear functions too simple for real-world problems
- Biological neurons also non-linear

Early Research (1943):

- Warren McCulloch and Walter Pitts
- Paper: “A logical calculus of the ideas immanent in nervous activity”
- First mathematical model of neurons

Learning Theory (1949):

- Donald Hebb
- Book: “The Organization of Behavior: A Neuropsychological Theory”
- Proposed how networks can be trained
- **Hebbian Learning**: “Cells that fire together, wire together”

Topics Covered:

① **Perceptron**

- One of the earliest neural network models
- Two-class classification

② **Multi-layered Networks**

- Building our own framework
- Understanding deep architecture

Topics Covered:

③ Neural Network Frameworks

- PyTorch
- Keras/TensorFlow
- Industry-standard tools

④ Overfitting

- Common problem in neural networks
- Prevention techniques
- Regularization methods

Section 1: Perceptron

What is a Perceptron?

- Earliest neural network model
- Binary classification
- Single-layer architecture
- Foundation for modern neural networks

Learn more: See Perceptron README

Section 2: Multi-layered Networks

Going Deeper:

- Multiple layers of neurons
- Hidden layers
- Building blocks of deep learning

Hands-on Practice:

- Code MLP Using SciKit-Learn for diabetes dataset
- Understand backpropagation
- See how training works internally

Section 3: Neural Network Frameworks

Modern Tools:

PyTorch

- Dynamic computational graphs
- Research-friendly
- Python-first approach
- Notebook: IntroPyTorch.ipynb

Keras/TensorFlow

- High-level API
- Production-ready
- Easy to use
- Notebook: IntroKerasTF.ipynb

Section 4: Overfitting

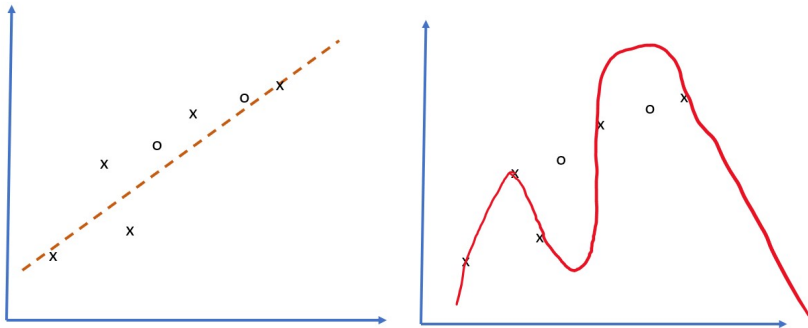
What is Overfitting?

- Model learns training data too well
- Poor generalization to new data
- Memorization vs. learning

Prevention Techniques:

- Regularization (L1, L2)
- Dropout
- Early stopping
- Data augmentation
- Cross-validation

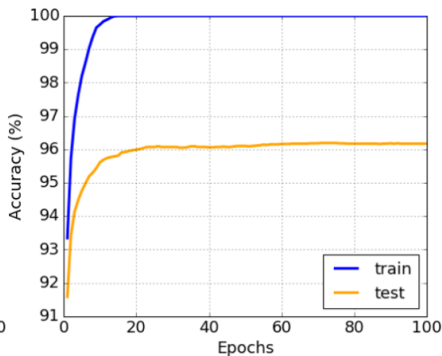
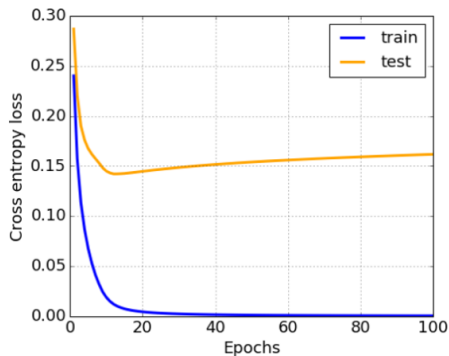
Overfitting Visualization



Signs of Overfitting:

- Training accuracy: Very high
- Validation accuracy: Much lower
- Model too complex for the data

Overfitting Diagram



Three Scenarios:

- **Underfitting:** Too simple, misses patterns
- **Good fit:** Captures patterns, generalizes well
- **Overfitting:** Too complex, captures noise

Key Concepts Summary

Neural Networks:

- Mathematical models inspired by the brain
- Consist of interconnected artificial neurons
- Learn from data through training

Core Components:

- Inputs with weights
- Activation functions
- Multiple layers
- Backpropagation for training

By the end of this section, you will:

- ① Understand neuron mathematical models
- ② Build neural networks from scratch
- ③ Use industry-standard frameworks
- ④ Recognize and prevent overfitting
- ⑤ Apply neural networks to real problems

Why Neural Networks?

Advantages:

- Learn complex non-linear patterns
- Automatic feature extraction
- Scalable to large datasets
- State-of-the-art in many domains

Applications:

- Image recognition
- Natural language processing
- Speech recognition
- Game playing (AlphaGo)
- Autonomous vehicles

Getting Started

Prerequisites:

- Basic Python programming
- Linear algebra fundamentals
- Understanding of calculus (derivatives)
- Familiarity with NumPy

Tools Needed:

- Python 3.x
- Jupyter Notebook
- PyTorch or TensorFlow
- NumPy, Matplotlib

From
Per-
cep-
tron

The Learning Process:

- ① **Initialize** weights randomly
- ② **Forward pass**: Compute predictions
- ③ **Calculate loss**: Measure error
- ④ **Backward pass**: Compute gradients
- ⑤ **Update weights**: Gradient descent
- ⑥ **Repeat** until convergence

Activation Functions

Common Functions:

- **Sigmoid:** $\sigma(x) = 1/(1+\exp(-x))$
 - Output: (0, 1)
 - Used in binary classification
- **ReLU:** $f(x) = \max(0, x)$
 - Most popular in hidden layers
 - Solves vanishing gradient problem
- **Softmax:** For multi-class classification
 - Outputs probability distribution

Neural Network Architectures

Types of Networks:

- **Feedforward**: Data flows in one direction
- **Convolutional (CNN)**: For images
- **Recurrent (RNN)**: For sequences
- **Transformer**: For language tasks
- **Generative Adversarial (GAN)**: For generation

This Section: Focus on feedforward networks

Practical Considerations

Model Design:

- Number of layers
- Number of neurons per layer
- Choice of activation functions
- Learning rate selection

Training Tips:

- Batch size selection
- Number of epochs
- Monitoring validation loss
- Using callbacks

Common Challenges

Training Issues:

- Vanishing/exploding gradients
- Overfitting (memorization)
- Underfitting (too simple)
- Slow convergence
- Getting stuck in local minima

Solutions:

- Proper initialization
- Batch normalization
- Regularization
- Learning rate scheduling
- Early stopping

Further Learning:

- [Perceptron README](#)
- [Multi-layered Networks README](#)
- [Neural Network Frameworks README](#)
- [Jupyter Notebooks in this directory](#)

External Resources:

- [Machine Learning for Beginners curriculum](#)
- [Deep Learning textbooks](#)
- [Research papers](#)
- [Online courses](#)

Next Steps

Continue Your Journey:

- ① Work through the Perceptron lesson
- ② Build your own neural network framework
- ③ Experiment with PyTorch and TensorFlow
- ④ Practice on real datasets
- ⑤ Explore advanced architectures

Remember:

- Practice is key to understanding
- Start simple, then increase complexity
- Experiment and learn from failures

Thank You!

Questions?

Let's dive into:

- Perceptron implementation
- Building neural networks
- Using modern frameworks
- Solving real problems

Next Lesson: Perceptron - The Foundation of Neural Networks

PART 2: The Perceptron

Introduction to Neural Networks: Perceptron

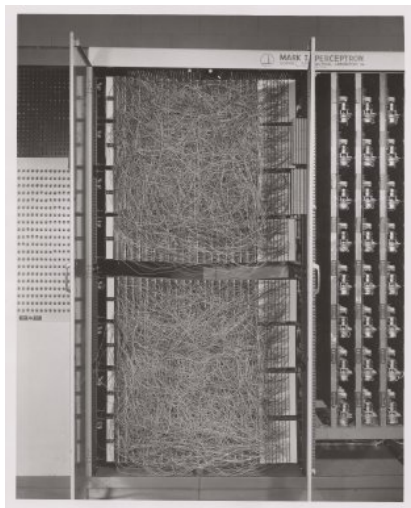
Frank Rosenblatt (1957)

- Cornell Aeronautical Laboratory
- First hardware implementation: “Mark-1”
- Designed to recognize primitive geometric figures
- Triangles, squares, and circles

Input Representation:

- 20x20 photocell array
- 400 inputs
- One binary output

The Mark-1 Perceptron



Images from Wikipedia

Simple Neural Network:

- Single neuron (threshold logic unit)
- Neural network weights as potentiometers
- Manual adjustment during training phase

Potentiometer: Device allowing user to adjust circuit resistance

The New York Times wrote:

“The embryo of an electronic computer that [the Navy] expects will be able to walk, talk, see, write, reproduce itself and be conscious of its existence.”

Reality vs. Expectations:

- Revolutionary for its time
- But had significant limitations
- Led to “AI Winter” in 1969

Binary Classification Model

- Distinguishes between two classes
- Input vector: x (size N features)
- Output: $+1$ or -1

Output Formula:

$$y(x) = f(w^T x)$$

where f is a step activation function

Step Activation Function

Function Definition:

$$f(x) = \begin{cases} +1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Characteristics:

- Binary output
- Sharp threshold at zero
- Non-differentiable (causes training challenges)

Function Definition:

$$f(x) = +1 \text{ if } x \geq 0$$

$$f(x) = -1 \text{ if } x < 0$$

Characteristics:

Training the Perceptron

Goal: Find weights vector w that: - Classifies most values correctly - Results in smallest error

Perceptron Criterion:

$$E(w) = -\text{sum}(w^T x_i t_i)$$

where: - Sum taken on misclassified training points - x_i = input data - t_i = -1 or +1 (target labels)

Gradient Descent

Optimization Method:

Start with initial weights $w^{(0)}$

Update at each step:

$$w^{(t+1)} = w^{(t)} - \eta * \nabla E(w)$$

where:

- η = learning rate
- $\nabla E(w)$ = gradient of error

Final Update Rule:

$$w^{(t+1)} = w^{(t)} + \text{sum}(\eta * x_i * t_i)$$

Training Algorithm Components

Key Elements:

- **Learning Rate (η):** Controls step size
- **Gradient:** Direction of steepest descent
- **Weight Update:** Adjust based on errors
- **Iteration:** Repeat until convergence

Process:

- ① Initialize weights
- ② Compute output
- ③ Calculate error
- ④ Update weights
- ⑤ Repeat

Python Implementation

```
def train(positive_examples, negative_examples,
          num_iterations = 100, eta = 1):

    weights = [0,0,0] # Initialize weights

    for i in range(num_iterations):
        pos = random.choice(positive_examples)
        neg = random.choice(negative_examples)

        z = np.dot(pos, weights)
        if z < 0: # positive misclassified
            weights = weights + eta*weights.shape

        z = np.dot(neg, weights)
        if z >= 0: # negative misclassified
            weights = weights - eta*weights.shape
```

Training Process Explained

Step-by-Step:

- ① **Initialize:** Random or zero weights
- ② **Select Samples:** Pick positive and negative examples
- ③ **Compute Output:** Calculate $z = w^T x$
- ④ **Check Classification:**
 - If positive classified as negative: increase weights
 - If negative classified as positive: decrease weights
- ⑤ **Iterate:** Repeat for num_iterations

Learning Rate Selection

eta (Learning Rate):

- **Too Large:**
 - Overshooting optimal weights
 - Oscillation, no convergence
- **Too Small:**
 - Slow convergence
 - Many iterations needed
- **Just Right:**
 - Steady progress
 - Efficient convergence

Typical Range: 0.001 to 0.1

Perceptron Limitations

Discovered by Minsky & Papert (1969):

Cannot solve XOR problem:

- Non-linearly separable data
- Need multiple layers
- Led to first “AI Winter”

Other Limitations:

- Only linear decision boundaries
- Sensitive to outliers
- No probabilistic outputs
- Single layer architecture

XOR Problem Illustration

Truth Table:

X1	X2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Problem: Cannot draw single straight line to separate classes

Solution: Multi-layer perceptron (MLP)

When Perceptrons Work Well

Linearly Separable Problems:

- Binary classification
- Simple pattern recognition
- Text classification (bag of words)
- Spam detection
- Simple geometric shapes

Advantages:

- Fast training
- Simple to implement
- Interpretable weights
- Foundation for understanding deep learning

Evolution:

- ① **1958:** Single-layer perceptron
- ② **1969:** Limitations discovered
- ③ **1986:** Multi-layer perceptron + backpropagation
- ④ **1990s:** Support Vector Machines dominate
- ⑤ **2006:** Deep learning renaissance
- ⑥ **2012+:** Modern deep neural networks

Multi-Layer Perceptron (MLP)

Solving Perceptron Limitations:

- Add hidden layers
- Use differentiable activation functions
- Backpropagation for training
- Can solve XOR and complex problems

Architecture:

- Input layer
- One or more hidden layers
- Output layer

Where Perceptrons Are Still Used:

① Linear Classifiers:

- Simple baseline models
- Feature importance analysis

② Ensemble Components:

- Part of voting systems
- Combination with other models

③ Educational Purposes:

- Understanding neural network basics
- Introduction to gradient descent

Perceptron vs. Modern Networks

Feature	Perceptron	Modern DNNs
Layers	Single	Multiple
Activation	Step	ReLU, Sigmoid, etc.
Training	Simple rule	Backpropagation
Capacity	Linear only	Non-linear
Speed	Very fast	Slower
Applications	Limited	Universal

Essential Concepts:

- ① First artificial neural network model
- ② Binary classification using weighted sum
- ③ Step activation function
- ④ Gradient descent training
- ⑤ Linear decision boundary limitation
- ⑥ Foundation for modern deep learning

Common Issues:

- **Not Converging:**
 - Adjust learning rate
 - Check data separability
 - Increase iterations
- **Poor Performance:**
 - Data not linearly separable
 - Need feature engineering
 - Consider MLP instead

Improving Performance:

- **Polynomial Features:** x, x^2, x^3
- **Interaction Terms:** $x_1 \cdot x_2$
- **Normalization:** Scale features
- **Domain Knowledge:** Create relevant features

Goal: Make data linearly separable

Understanding Perceptron:

- ① **Decision Boundary:** Plot separating line
- ② **Weight Vector:** Direction of classification
- ③ **Training Progress:** Error over iterations
- ④ **Feature Space:** 2D/3D projections

Tools:

- Matplotlib for plotting
- Seaborn for visualization
- Interactive plots with Plotly

Perceptron Convergence Theorem

Theoretical Guarantee:

If data is linearly separable:

- Perceptron will converge
- Finite number of iterations
- Find perfect separator

Conditions:

- Data must be linearly separable
- Finite dataset
- Proper learning rate

Extensions and Improvements:

① **Voted Perceptron:**

- Multiple weight vectors
- Voting scheme

② **Averaged Perceptron:**

- Average all weight vectors
- Better generalization

③ **Kernel Perceptron:**

- Non-linear decision boundaries
- Kernel trick application

What the Hands-On Scripts Add:

- Load a real dataset
- Choose features and labels
- Split data into training and testing sets
- Train a model with scikit-learn
- Predict on unseen test data
- Evaluate and visualize the result

Key Idea: Practice the full ML loop, not only the model formula

Why split the data?

- Training set: used to fit the model
- Test set: used to estimate performance on new data
- Prevents evaluating only on examples the model already saw

Practical Details:

- `random_state` makes examples reproducible
- `stratify` preserves class balance for classification
- Test size controls how much data is held out

Linear Regression: Diabetes Example

Regression Goal:

- Predict a continuous numerical value
- Example: diabetes progression score

Model Idea:

$$\hat{y} = wx + b$$

What Students Practice:

- Select one feature for 2D visualization
- Fit a linear model
- Compare actual values and predictions
- Interpret prediction error visually

Classification: Breast Cancer Example

Classification Goal:

- Predict a discrete class label
- Example: malignant or benign tumor

Gaussian Naive Bayes:

- Probabilistic classifier
- Assumes features are conditionally independent
- Models each feature distribution with a Gaussian curve
- Fast baseline for small tabular datasets

Metric: $\text{Accuracy} = \text{correct predictions} / \text{all predictions}$

Why Explain Models?

- Accuracy alone does not explain a decision
- Healthcare examples need interpretable reasoning
- Explanations help check whether the model uses sensible features

SHAP Values:

- Estimate each feature's contribution to a prediction
- Positive values push prediction higher
- Negative values push prediction lower
- Summary plots show global feature importance

Generated Artifacts:

- Diabetes regression plot: `1_diabetes_1_plot.png`
- Breast cancer class distribution plot
- SHAP summary plot
- SHAP dependence plot for BMI
- SHAP force plots saved as HTML

Practical Note:

- Interactive backends display figures with `plt.show()`
- Headless backends such as Agg save figures to files

Microsoft Learn Lab:

- Build perceptron with Azure ML
- Two-class averaged perceptron
- Azure ML Designer interface

Additional Reading:

- “What is a Perceptron: Basics of Neural Networks”
- Towards Data Science articles
- Original Rosenblatt papers

Summary and Conclusion

Perceptron Legacy:

- Historical significance in AI
- Foundation of neural networks
- Still relevant for education
- Simple yet powerful concept

Next Steps:

- Multi-layer networks
- Backpropagation
- Modern frameworks (PyTorch, TensorFlow)

Why Study Perceptrons?

- ① Understand neural network fundamentals
- ② Grasp gradient descent concepts
- ③ Appreciate deep learning evolution
- ④ Build intuition for complex models

Remember:

- Every deep network is built on these principles
- Simple models teach important lessons
- Start simple, scale complexity gradually

Continue Learning!